
Progetto Mesos — am25

Analisi del Protocollo di Comunicazione

Anton Robert, Bortolazzi Filippo, Caliment Sara, Delgrossi Daniele

Il presente documento descrive in dettaglio il protocollo di comunicazione adottato nel gioco da tavolo digitale *Mesos*. Per ciascuna fase vengono illustrati i messaggi scambiati, i componenti coinvolti, le motivazioni delle scelte progettuali e le sequenze di interazione tra client e server, documentate tramite diagrammi UML estratti direttamente dal progetto.

Sequenze documentate	Messaggi totali	FA coperte
13	40	3

Indice

Panoramica	3
1 Architettura di rete	5
1.1 Componenti client	5
1.2 Componenti server	5
1.3 Meccanismo dello stub RMI in-band	5
2 Gerarchia dei messaggi	7
2.1 Messaggi Client → Server	7
2.2 Messaggi Server → Client	7
2.3 Dispatch polimorfico lato server e lato client	7
3 Lobby e setup	11
3.1 Notifiche durante la fase di accumulo	11
3.2 setupAndStartGame() e legame degli observer	11
3.3 forceInitialPlayersSync: ridondanza intenzionale	11
4 Fase di piazzamento	14
4.1 Loop di piazzamento	14
4.2 Transizione a RESOLVE_ACTION	14
5 Fase di gioco	16
5.1 Selezione carte tribù e acquisto edifici	16
5.2 Gestione azioni disponibili e avanzamento turno	16
6 Fine round: risoluzione eventi e aggiornamento mercato	17
6.1 Ritorno ai default tile e OrderOnDefaultTileMessage	17
6.2 Risoluzione degli eventi	17
6.3 Aggiornamento del mercato e cambio era	17
6.4 Salvataggio atomico a fine round	17
7 Extra Draw	19
7.1 Snapshot pre-refresh e invio unicast	20
7.2 Selezione per reference equality nelle liste live	20
7.3 Skip e no-op	20
8 Fine partita	21
8.1 Calcolo del punteggio finale	21
8.2 Notifica vincitori e transizione END_GAME	22
8.3 Richiesta classifica: unicast post-partita	22

9 Heartbeat — Ping/Pong	23
9.1 Ping client → server	23
9.2 Pong server → client e Heartbeat Watchdog	23
9.3 PongWatchdog lato client	23
10 Disconnessione	25
10.1 I tre percorsi di rilevamento	26
10.2 Handler sincronizzato e effetti sulla partita	26
11 Riconnessione	27
11.1 Rilevamento e swap dello stub	27
11.2 Grace window sul watchdog	28
11.3 Resync completo dello stato	28
12 Caricamento partita salvata	29
12.1 Fase 1: il primo giocatore carica la partita	29
12.2 Fase 2: i giocatori rimanenti si uniscono	29
12.3 Fase 3: startLoadedGame() e resumeGame()	29
13 Database e classifica	32
13.1 Schema del database	32
13.2 Registrazione del risultato: logMatch()	32
13.3 Recupero della classifica: getLeaderboard()	32
Considerazioni conclusive	34
A Architettura di rete — diagramma completo	36
B Gerarchia completa dei messaggi	37
C Fase di gioco — diagramma completo	38
D Fine round — risoluzione eventi: diagramma completo	39

Panoramica

Contesto del progetto

Il progetto consiste nello sviluppo di una versione software del gioco da tavolo *Mesos*, realizzata nell'ambito della Prova Finale di Ingegneria del Software del Politecnico di Milano (A.A. 2025–2026). Il sistema è distribuito con architettura **client-server** e supporta due protocolli di trasporto alternativi: **Java RMI** e **Socket TCP/IP**. La scelta del trasporto è demandata al client all'avvio, mentre l'intera logica applicativa rimane invariata grazie al pattern Proxy (descritto nel report di analisi dei design pattern).

Funzionalità avanzate documentate

In aggiunta al protocollo di base, il documento tratta tre funzionalità avanzate (FA) la cui corretta implementazione richiede flussi di messaggi dedicati:

- **Persistenza (FA)** — salvataggio periodico dello stato della partita su disco al termine di ogni round; ripristino integrale tramite una sessione di caricamento coordinata (Capitolo 12);
- **Resilienza alle disconnessioni (FA)** — rilevamento multi-percorso delle disconnessioni, gestione del turno in assenza del giocatore e rientro in partita con risincronizzazione completa dello stato (Capitoli 10 e 11);
- **Classifica su DB (FA)** — registrazione asincrona dei risultati su database AWS RDS MySQL e recupero unicast della classifica a fine partita (Capitolo 13).

Struttura del protocollo

Il protocollo è organizzato in due direzioni di comunicazione simmetriche, ciascuna modellata come una famiglia di oggetti **Serializable** che si auto-dispacciano tramite il pattern Command:

- **Client** → **Server**: messaggi di controllo inviati dal client al **ServerNetworkHandler** tramite **ClientToServerMessage**;
- **Server** → **Client**: notifiche di stato inviate dalla **ServerVirtualView** al client tramite **ServerToClientMessage** (Socket) o chiamata RMI diretta sull'interfaccia **ClientRemoteInterface**.

Il transport layer è completamente trasparente al codice applicativo: con Socket i messaggi vengono serializzati su **ObjectOutputStream**; con RMI le stesse operazioni corrispondono a invocazioni di metodo remoto sull'interfaccia condivisa.

La tabella seguente riassume i tredici argomenti trattati nel documento, con la classificazione adottata e le classi chiave coinvolte.

#	Argomento	Tipo	Classi principali
1	Architettura di rete	Strutturale	ServerNetworkHandler, ClientVirtualView
2	Gerarchia dei messaggi	Strutturale	ClientToServerMessage, ServerToClientMessage
3	Lobby e setup	Sequenziale — Inizializ.	ServerNetworkHandler, ServerVirtualView
4	Fase di piazzamento	Sequenziale — Gameplay	PlacingPlayerMessage, Board
5	Fase di gioco	Sequenziale — Gameplay	SelectCard*Message, Market
6	Risoluzione eventi e mercato	Sequenziale — Gameplay	Market, ResolvedEventMessage
7	Extra Draw	Sequenziale — Gameplay	AskExtraDrawMessage, SelectExtraCardMessage
8	Fine partita	Sequenziale — Terminaz.	GameWinnersMessage, DBManager
9	Heartbeat	Infrastrutturale — Resil.	PingMessage, PongMessage, PongWatchdog
10	Disconnessione	Infrastrutturale — Resil.	PlayerDisconnectedMessage
11	Riconnessione	Infrastrutturale — Resil.	PlayerReconnectedMessage, ServerVirtualView
12	Caricamento partita salvata	Infrastrutturale — Persist.	LoadGameMessage, PersistenceLogger
13	Database e classifica	Infrastrutturale — Persist.	DBManager, SendRankMessage

1. Architettura di rete

Tipo: Strutturale — Panoramica

Descrizione e motivazione della scelta

Il sistema è articolato in due JVM distinte, client e server, collegate tramite un doppio canale di trasporto alternativo: Socket TCP/IP e Java RMI. Si è scelto di rendere il trasporto completamente trasparente al codice applicativo mediante il pattern Proxy: sia il server che il client operano sempre contro interfacce comuni (**ServerRemoteInterface** e **ClientRemoteInterface**), senza alcuna ramificazione condizionale sul protocollo nel codice di gioco.

Allegato 1 — `allegato_01_architettura_di_rete.pdf`

Panoramica completa dell'architettura con Client JVM, Server JVM, canale TCP/IP e registro RMI, con tutte le dipendenze tra componenti.

1.1 Componenti client

Sul lato client operano quattro componenti principali. La **View** (TUI o GUI) è il layer di presentazione che interagisce con l'utente. Il **ClientApp** funge da launcher e da punto di ingresso per tutte le azioni di gioco: chiama il metodo corrispondente sull'interfaccia **ServerRemoteInterface**, indifferente che il trasporto sottostante sia RMI o Socket.

Per ricevere le notifiche del server sono presenti due implementazioni di **ClientRemoteInterface**, una per ciascun trasporto: **ServerListener** (Socket), un thread daemon che deserializza gli oggetti **ServerToClientMessage** in arrivo e invoca `execute()` su ciascuno; **ClientVirtualView** (RMI), un oggetto che estende **UnicastRemoteObject** e viene esportato automaticamente all'avvio, ricevendo le chiamate di push direttamente dal server. Entrambi aggiornano la View e notificano il **PongWatchdog**, che monitora la liveness del server lato client.

1.2 Componenti server

Sul lato server il **ServerNetworkHandler** è l'oggetto RMI esportato nel registro: riceve tutte le chiamate dei client (sia via RMI che, indirettamente, via Socket attraverso lo **SocketClientHandler**) e le delega al **Controller**. Per ciascun client connesso esiste una **ServerVirtualView** dedicata, che implementa tutti e quattro gli observer del modello (**GameObserver**, **BoardObserver**, **MarketObserver**, **PlayerObserver**) e mantiene un snapshot locale completo dello stato di gioco, necessario per la risincronizzazione in caso di riconnessione (Capitolo 11).

1.3 Meccanismo dello stub RMI in-band

Con il trasporto RMI il server non recupera mai lo stub del client dal registro: è il client a passarlo *in-band* come parametro nelle chiamate `createGame(PlayerDT0, int,`

`ClientRemoteInterface`) e `addPlayer(PlayerDT0, ClientRemoteInterface)`. La `ClientVirtualView` viene esportata automaticamente tramite `super()` del costruttore di `UnicastRemoteObject`, per cui il client può passare `this` come stub valido già prima che il server abbia mai tentato di contattarlo. Il server memorizza lo stub ricevuto nel campo `ServerVirtualView.clientStub` e lo utilizza per tutte le notifiche successive, senza ulteriori lookup nel registro.

Classi e messaggi di riferimento

```
server/webLayer/RMI/ServerNetworkHandler.java
server/webLayer/RMI/ClientRemoteInterface.java
server/webLayer/RMI/ServerRemoteInterface.java
server/webLayer/Socket/ClientSocketProxy.java
server/webLayer/Socket/SocketClientHandler.java
server/webLayer/ServerVirtualView.java
client/webLayer/RMI/ClientVirtualView.java
client/webLayer/Socket/ServerListener.java
client/webLayer/PongWatchdog.java
```

2. Gerarchia dei messaggi

Tipo: Strutturale — Messaggistica

Descrizione e motivazione della scelta

Il protocollo Socket è interamente basato sul pattern Command: ogni operazione di rete corrisponde a un oggetto `Serializable` autonomo che incapsula i propri dati e sa come eseguirsi, invocando il metodo corretto sull'interfaccia remota di destinazione. Si è scelto questo approccio per rendere il dispatcher completamente generico e privo di `switch/instanceof`: l'aggiunta di un nuovo tipo di messaggio non richiede alcuna modifica al codice di ricezione. La gerarchia completa è riportata nell'Allegato A.

2.1 Messaggi Client → Server

L'interfaccia `ClientToServerMessage` definisce il contratto comune per tutte le azioni che il client può inviare al server. I tredici tipi di messaggio sono raggruppati per funzione:

Categoria	Messaggi
Lobby	<code>CreateGameMessage</code> , <code>AddPlayerMessage</code>
Persistenza	<code>LoadGameMessage</code> , <code>JoinGameLoadedMessage</code>
Piazzamento	<code>PlacingPlayerMessage</code>
Gameplay	<code>SelectCardFromTopListMessage</code> , <code>SelectCardFromBottomListMessage</code> , <code>PlayerDoNothingMessage</code>
Extra Draw	<code>SelectExtraCardMessage</code> , <code>SkipExtraDrawMessage</code>
Post-partita	<code>AskForRankMessage</code>
Heartbeat	<code>PingMessage</code>
Errore	<code>ErrorMessage</code>

2.2 Messaggi Server → Client

L'interfaccia `ServerToClientMessage` definisce il contratto comune per le ventisette notifiche inviate dal server ai client. Queste notifiche, a differenza dei messaggi $C \rightarrow S$, sono sempre *broadcast*: la `ServerVirtualView` di ogni client connesso riceve la stessa chiamata e la propaga al proprio stub. Fanno eccezione `AskExtraDrawMessage` e `SendRankMessage`, che sono *unicast* in quanto destinate a un solo giocatore.

2.3 Dispatch polimorfico lato server e lato client

Lato server, il `SocketClientHandler` deserializza ogni `ClientToServerMessage` e invoca `execute(server, proxy)` senza conoscerne il tipo concreto. Lato client, il `ServerListener` fa lo stesso con

`ServerToClientMessage.execute(client)`. L'intera logica di instradamento è quindi delegata agli oggetti messaggio stessi, non ai dispatcher: aggiungere un nuovo tipo di messaggio richiede solo una nuova classe, senza toccare il codice di ricezione esistente.

Classi e messaggi di riferimento

```
server/webLayer/Socket/ServerToClientMessage.java  
client/webLayer/Socket/ClientToServerMessage.java  
server/webLayer/Socket/SocketClientHandler.java  
client/webLayer/Socket/ServerListener.java
```

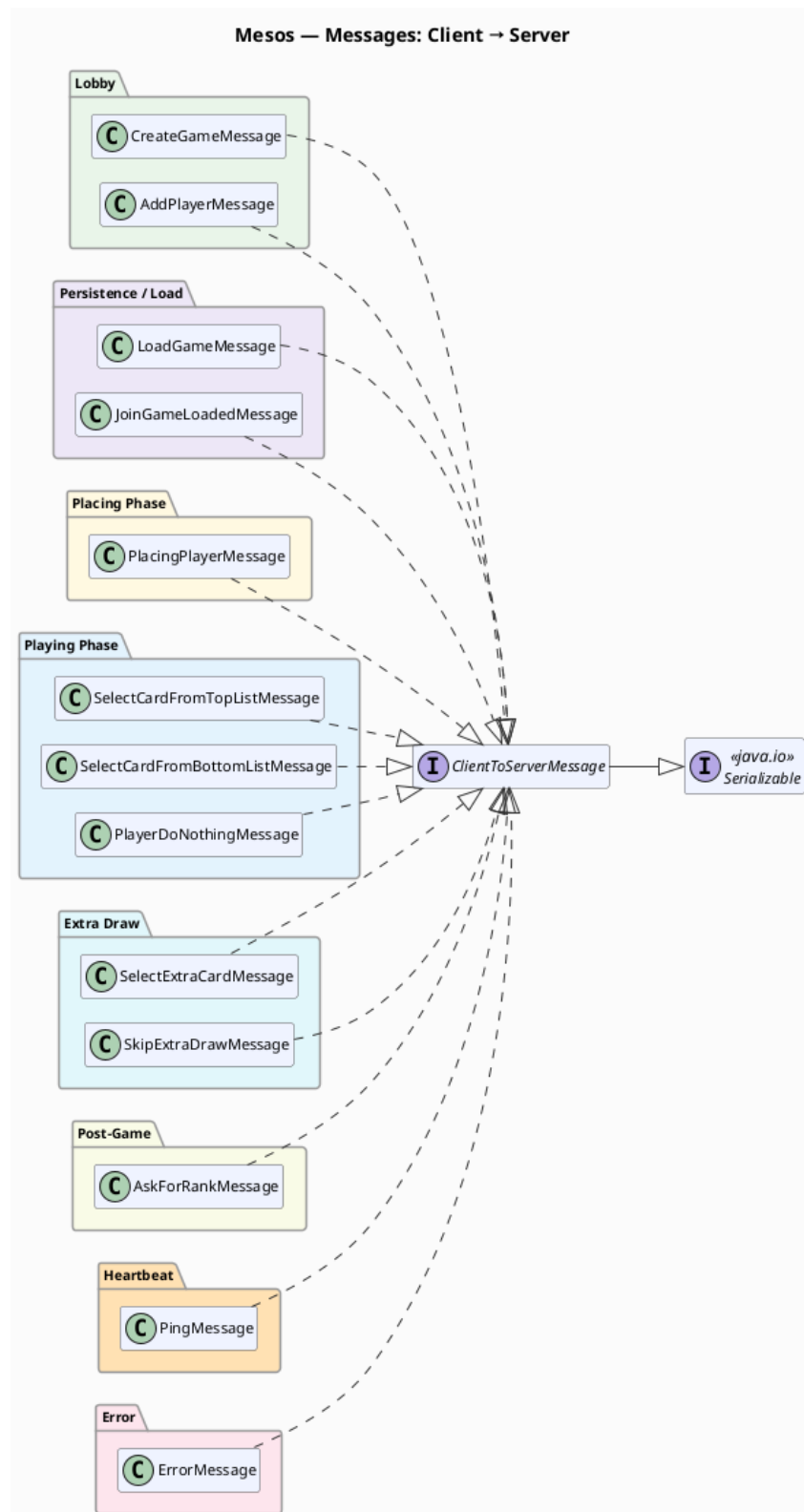


Figura 2.1: Gerarchia dei messaggi Client → Server.
 Allegato 2 — allegato_02_messaggi_client_server.pdf

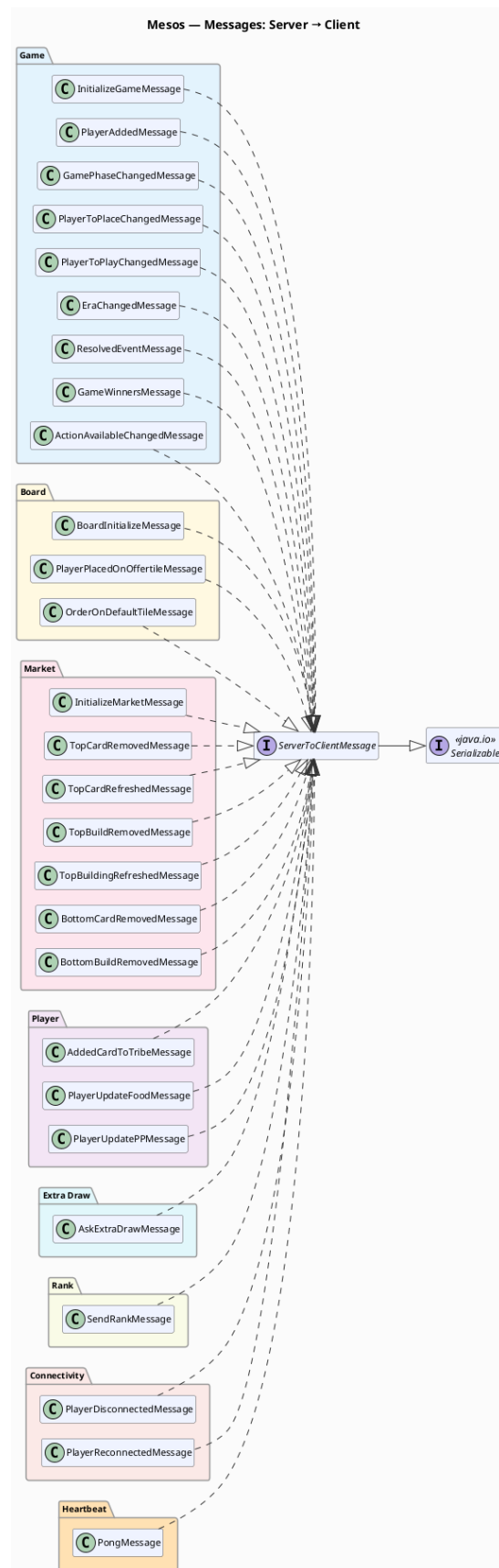


Figura 2.2: Gerarchia dei messaggi Server → Client.
 Allegato 3 — allegato_03_messaggi_server_client.pdf

3. Lobby e setup

Tipo: Sequenziale — Inizializzazione

Descrizione e motivazione della scelta

La fase di lobby gestisce l'accumulo dei giocatori prima che il modello di dominio esista. Si è scelto di separare nettamente la fase di *buffering* dalla fase di *avvio*: il `ServerNetworkHandler` accumula le `ServerVirtualView` nella lista `waitingPlayers` e, solo quando l'ultimo giocatore entra, chiama `setupAndStartGame()` che crea il modello, lega gli observer e avvia la partita in un'unica sequenza atomica `synchronized`.

3.1 Notifiche durante la fase di accumulo

Al contrario di quanto un approccio naive potrebbe suggerire, i messaggi *non* vengono bufferizzati in attesa dell'avvio del modello: la fase di lobby è già attiva nella comunicazione. Ogni volta che un giocatore si connette, il `ServerNetworkHandler` chiama `pushPlayerAdded()` per aggiornare sia il client appena arrivato sia tutti i client già connessi:

- il **nuovo giocatore** riceve `PlayerAddedMessage` per ogni giocatore già in lobby, incluso se stesso;
- ogni **giocatore esistente** riceve `PlayerAddedMessage` per il nuovo arrivato.

`pushPlayerAdded()` aggiorna contestualmente il campo `playersMap` della `ServerVirtualView`, che rappresenta lo snapshot server-side dei giocatori e viene consultato durante `resyncClient()` in caso di riconnessione successiva (Capitolo 11).

3.2 `setupAndStartGame()` e legame degli observer

Quando `waitingPlayers.size() == requiredPlayers`, `setupAndStartGame()` esegue i seguenti passi in sequenza: crea il `Controller` e il modello di dominio; lega tutte le `ServerVirtualView` come observer del modello tramite `controller::linkObserver`; aggiunge i giocatori non-host al modello; chiama `crossRegisterPlayerObservers()` per garantire che le variazioni della tribù di ciascun giocatore siano broadcast a tutti i client; avvia il watchdog heartbeat; infine chiama `controllerGameStar()` che produce la sequenza di messaggi iniziali verso tutti i client.

3.3 `forceInitialPlayersSync`: ridondanza intenzionale

Al termine di `setupAndStartGame()`, prima di avviare la partita, si chiama `forceInitialPlayersSync(allPlayers)` su ogni `ServerVirtualView`. Questo reinvia a ogni client un `PlayerAddedMessage` per ciascun giocatore, inclusi quelli già inviati durante la fase di accumulo. Si è scelto di accettare questa ridondanza deliberatamente per due ragioni: garantire che il `playersMap` server-side di ogni view sia completo e coerente prima dell'avvio, indipendentemente dall'ordine di esecuzione dei thread durante la lobby; e assicurare che ogni client, anche in presenza di condizioni di race durante la connessione, riceva la lista completa dei giocatori prima di

qualsiasi messaggio di stato del modello. Il client gestisce i duplicati in modo idempotente tramite `ConcurrentHashMap.put()`.

Classi e messaggi di riferimento

<code>server/webLayer/RMI/ServerNetworkHandler.java</code>	<code>(createGame, addPlayer, setupAndStartGame)</code>
<code>server/webLayer/ServerVirtualView.java</code>	<code>(pushPlayerAdded, forceInitialPlayersSync)</code>
<code>server/webLayer/Socket/messages/AddPlayerMessage.java</code>	
<code>server/webLayer/Socket/messages/gameMessages/PlayerAddedMessage.java</code>	

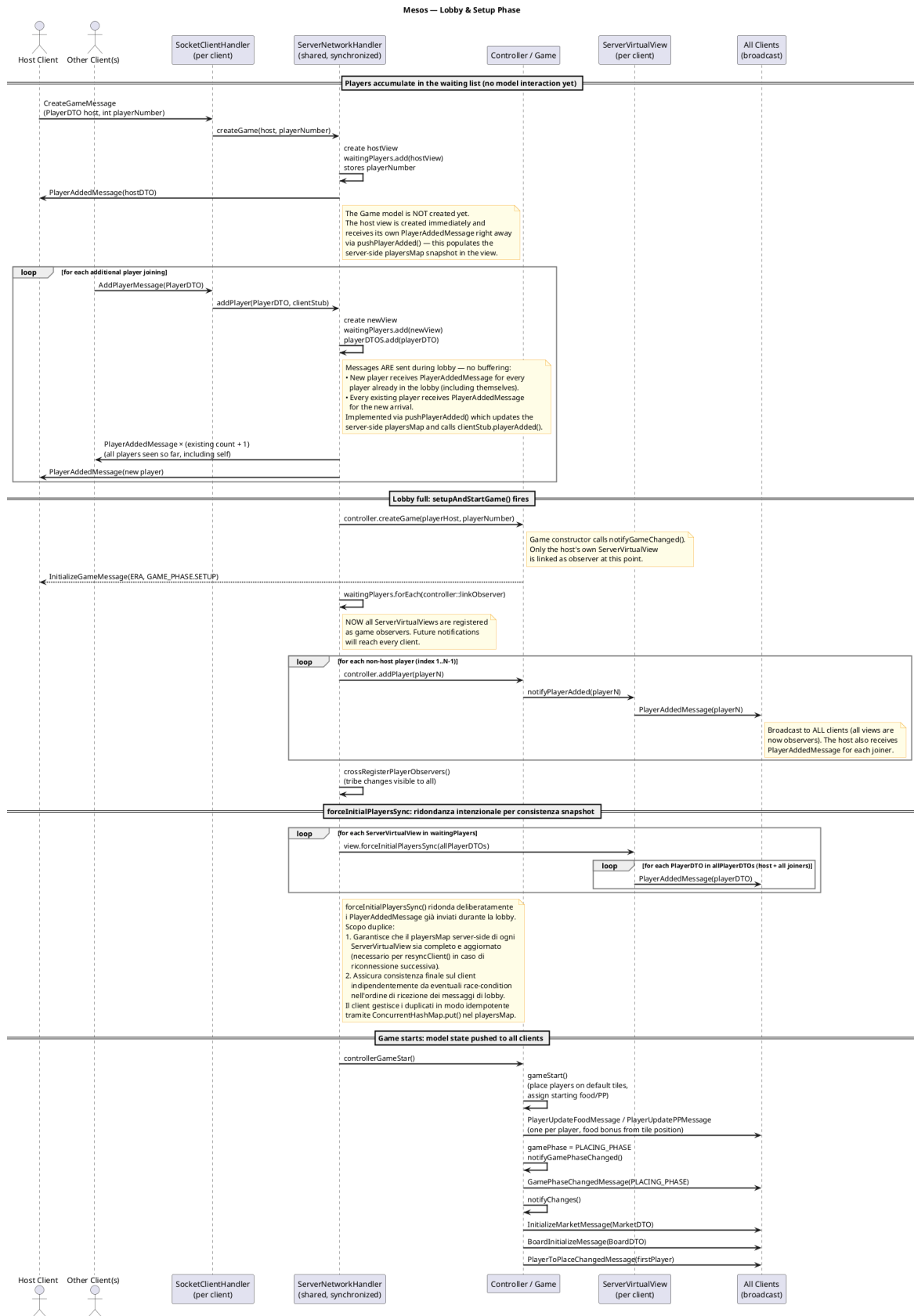


Figura 3.1: Sequenza di lobby e setup della partita.
Allegato 6 — allegato_06_lobby_e_setup.pdf

4. Fase di piazzamento

Tipo: Sequenziale — Gameplay

Descrizione e motivazione della scelta

La fase di piazzamento gestisce il posizionamento dei totem sulle offer tile prima di ogni round di gioco. Si è scelto di strutturarla come un semplice loop sequenziale: il server comunica al client corrente che è il suo turno tramite `PlayerToPlaceChangedMessage`, attende la risposta (`PlacingPlayerMessage`), aggiorna la board e notifica tutti i client. La struttura è volutamente lineare, priva di branch alternativi, perché il piazzamento non prevede decisioni condizionali: ogni giocatore sceglie una posizione libera e prosegue.

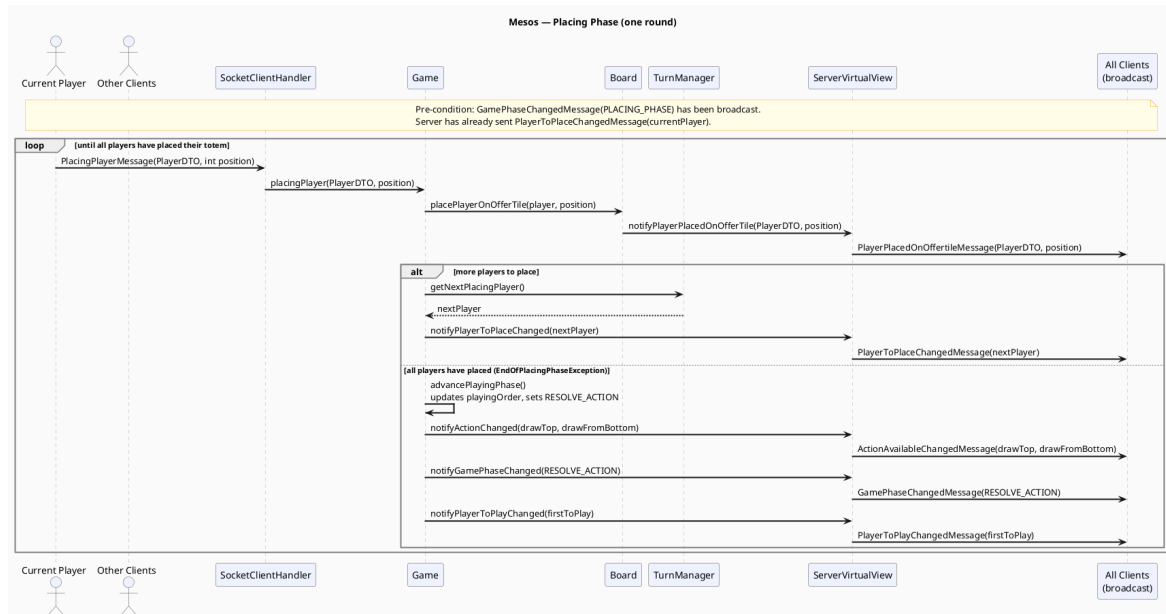


Figura 4.1: Sequenza della fase di piazzamento. Allegato 7 — allegato_07_fase_piazzamento.pdf

4.1 Loop di piazzamento

Il loop itera fino a quando tutti i giocatori hanno collocato il proprio totem. Ad ogni iterazione:

1. il client invia `PlacingPlayerMessage(PlayerDTO, position)`;
2. il server chiama `board.placePlayerOnOfferTile(player, position)`;
3. la board notifica gli observer: `ServerVirtualView` invia `PlayerPlacedOnOfferTileMessage` in broadcast;
4. se rimangono giocatori da piazzare, il server invia `PlayerToPlaceChangedMessage` con il giocatore successivo.

4.2 Transizione a `RESOLVE_ACTION`

All'esaurimento dei piazzamenti, il modello lancia internamente un'eccezione `EndOfPlacingPhaseException` che causa la transizione alla fase successiva. Il server invia in broadcast: `ActionAvailableChangedMessage`

con le azioni disponibili determinate dall'offer tile scelta; `GamePhaseChangedMessage(RESOLVE_ACTION)`; `PlayerToPlayChangedMessage` con il primo giocatore di turno. Le azioni disponibili (*drawTop* e *drawFromBottom*) dipendono dalla posizione dell'offer tile scelta.

Classi e messaggi di riferimento

```
client/webLayer/Socket/messages/PlacingPlayerMessage.java
server/webLayer/Socket/messages/gameMessages/PlayerPlacedOnOffertileMessage.java
server/webLayer/Socket/messages/gameMessages/PlayerToPlaceChangedMessage.java
server/webLayer/Socket/messages/gameMessages/ActionAvailableChangedMessage.java
server/model/Board/Board.java
```

5. Fase di gioco

Tipo: Sequenziale — Gameplay

Descrizione e motivazione della scelta

La fase di gioco (`RESOLVE_ACTION`) gestisce la selezione delle carte e degli edifici dal mercato da parte di ciascun giocatore. Si è scelto di modellare ogni variante di azione (carta da top, carta da bottom, edificio da top, edificio da bottom, passaggio) come ramo distinto di un `alt` nel diagramma di sequenza, mantenendo un'unica infrastruttura di conteggio delle azioni (`subtractAction()`). La sequenza completa è riportata nell'Allegato B.

5.1 Selezione carte tribù e acquisto edifici

Il client invia `SelectCardFromTopListMessage` o `SelectCardFromBottomListMessage`, specificando il tipo di carta (`CARD_TYPE`) e la posizione nella lista. Se il tipo è `BUILDING`, il server chiama `market.buyBuildingTopList()` o `buyBuildingBottomList()`: il giocatore paga il costo in cibo (`PlayerUpdateFoodMessage`) e riceve l'edificio (`AddedCardToTribeMessage`). In tutti gli altri casi il server chiama `market.selectCardFromTopList()` o `selectCardFromBottomList()`: la carta viene aggiunta alla tribù senza costo. In entrambi i casi il server notifica la rimozione dal mercato (`TopCardRemovedMessage`, `BottomCardRemovedMessage`, ecc.).

5.2 Gestione azioni disponibili e avanzamento turno

Dopo ogni azione, il server decrementa il contatore interno e invia `ActionAvailableChangedMessage` in broadcast con i nuovi valori di `drawTop` e `drawFromBottom`. Quando entrambi i contatori raggiungono zero, o quando il giocatore invia `PlayerDoNothingMessage` (caso in cui il mercato è bloccato), il server avanza al giocatore successivo. Se tutti i giocatori hanno esaurito le azioni, il server chiama `nextRoundIter()` (vedi Capitolo 6).

Classi e messaggi di riferimento

```
client/webLayer/Socket/messages/SelectCardFromTopListMessage.java
client/webLayer/Socket/messages/SelectCardFromBottomListMessage.java
client/webLayer/Socket/messages/PlayerDoNothingMessage.java
server/webLayer/Socket/messages/marketMessages/TopCardRemovedMessage.java
server/webLayer/Socket/messages/marketMessages/BottomCardRemovedMessage.java
server/webLayer/Socket/messages/playerMessage/AddedCardToTribeMessage.java
server/model/Game/Market.java
```

6. Fine round: risoluzione eventi e aggiornamento mercato

Tipo: Sequenziale — Gameplay

Descrizione e motivazione della scelta

A fine di ogni round il server esegue una sequenza articolata di operazioni: ritorno dei totem ai default tile, risoluzione degli eventi nella riga inferiore del mercato, aggiornamento del mercato con potenziale cambio era, e salvataggio dello stato su disco. Si è scelto di raggruppare queste operazioni in `nextRoundIter()` per garantire atomicità rispetto agli observer: nessun client riceve una notifica parziale di fine round. La sequenza completa è riportata nell'Allegato C.

6.1 Ritorno ai default tile e `OrderOnDefaultTileMessage`

Il primo passo di `nextRoundIter()` è `board.returnOnDefaultTiles()`, che sposta tutti i totem dalle offer tile ai default tile secondo l'ordine di piazzamento del round. Al termine, la board chiama `notifyPlayerToDefaultTile(playerOrder)`: la `ServerVirtualView` serializza l'ordine in una lista di `PlayerDTO` e invia `OrderOnDefaultTileMessage` in broadcast a tutti i client. Questo messaggio ha doppia funzione: aggiornare la visualizzazione della board e comunicare ai client l'ordine di piazzamento del round successivo.

6.2 Risoluzione degli eventi

Gli eventi presenti nella `bottomCardList` del mercato vengono risolti nell'ordine:

HUNT → PAINTINGS → SHAMANIC_RIT → SUSTENANCE. Per ciascun evento il server applica l'effetto su tutti i giocatori, inviando `PlayerUpdateFoodMessage` e/o `PlayerUpdatePPMessage` per ogni variazione, e infine `ResolvedEventMessage(eventID, eventType)` in broadcast. L'ordine di risoluzione è dettato dalle regole del gioco fisico.

6.3 Aggiornamento del mercato e cambio era

Dopo la risoluzione degli eventi, `endOfRoundMarketActions()` esegue in sequenza: azzeramento della `bottomCardList`, spostamento del `topCardList` nella lista inferiore (shift), ricarica del `topCardList` dal mazzo. Se il mazzo contiene carte di un'era successiva rispetto a quella corrente, si rileva un cambio era: `gameView.nextEra()` viene chiamato internamente e il server invia `EraChangedMessage`, seguito da `TopCardRefreshedMessage` e `TopBuildingRefreshedMessage` (gli edifici vengono anch'essi ruotati). Se il mazzo è esaurito, viene lanciata `DeckFinishedException` e la fase successiva diventa `LAST_ROUND_PLACING_PHASE`.

6.4 Salvataggio atomico a fine round

Al termine di `nextRoundIter()`, il Controller invoca `persistanceLogger.save(game.createMemento())`. Il salvataggio avviene su file temporaneo con successivo `ATOMIC_MOVE` per garantire l'integrità del file in caso di crash durante la scrittura. Il salvataggio è effettuato solo se `nextRoundIter()` è completato senza eccezione `EndGameException`: non si salva mai lo stato dell'ultimo round.

Classi e messaggi di riferimento

```
server/model/Game/Market.java (endOfRoundMarketActions, solveEvents)
server/model/Board/Board.java (returnOnDefaultTiles,
notifyPlayerToDefaultTile)
server/webLayer/Socket/messages/boardMessaes/OrderOnDefaultTileMessage.java
server/webLayer/Socket/messages/gameMessages/ResolvedEventMessage.java
server/webLayer/Socket/messages/marketMessages/TopCardRefreshedMessage.java
server/webLayer/Socket/messages/gameMessages/EraChangedMessage.java
server/model/persistence/PersistenceLogger.java
```

7. Extra Draw

Tipo: Sequenziale — Gameplay

Descrizione e motivazione della scelta

L'effetto edificio *DrawOneMoreCard* consente al proprietario di pescare una carta aggiuntiva dal mercato al termine di ogni round, prima che il mercato venga aggiornato. Si è scelto di inviare al giocatore uno snapshot del mercato nello stato pre-refresh, così che possa selezionare una carta dalla stessa offerta che ha visto durante il suo turno ordinario, indipendentemente dal momento in cui risponde.

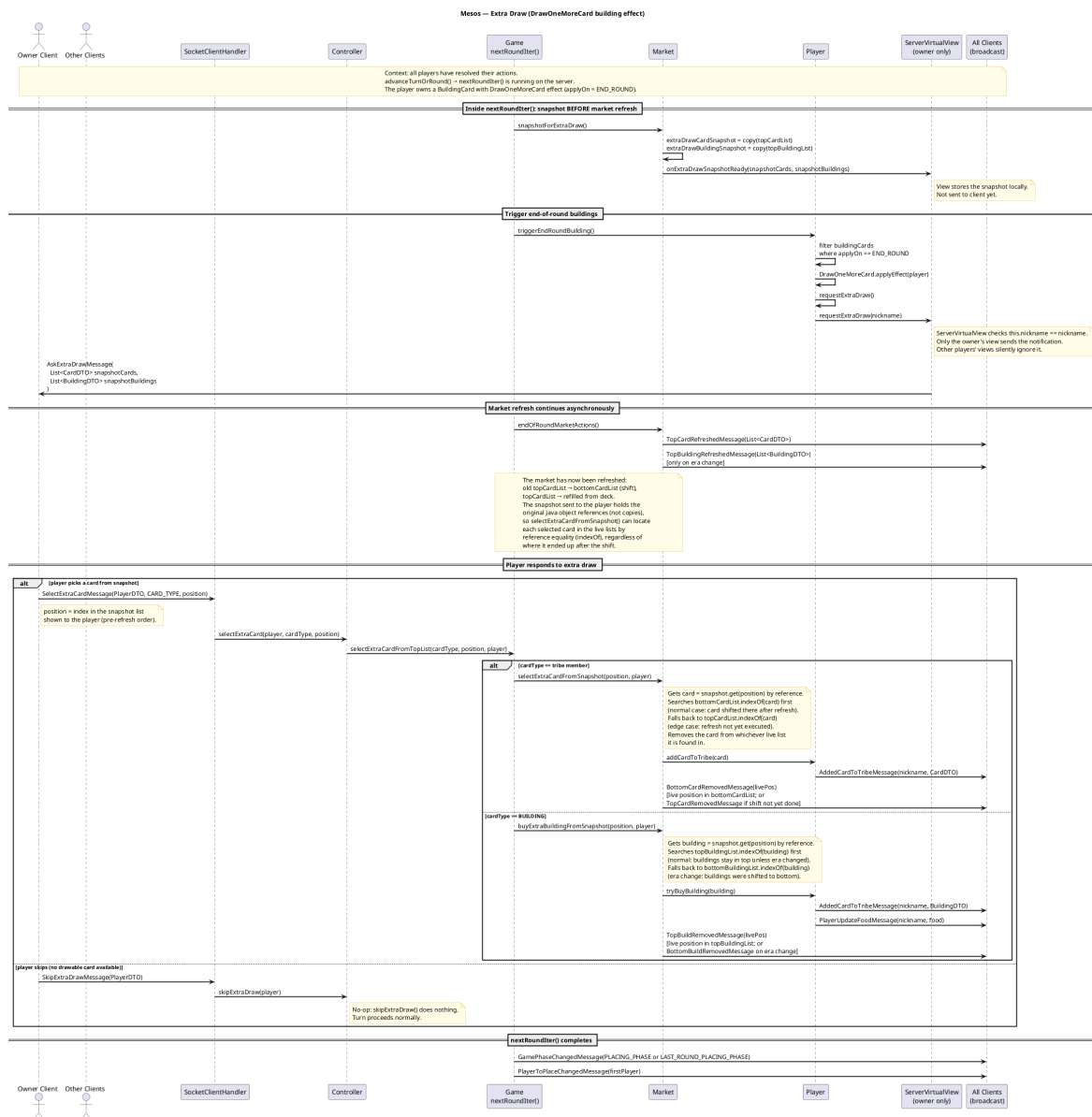


Figura 7.1: Sequenza dell'Extra Draw. Allegato 10 — allegato_10_extra_draw.pdf

7.1 Snapshot pre-refresh e invio unicast

All'interno di `nextRoundIter()`, prima di chiamare `endOfRoundMarketActions()`, il server invoca `market.snapshotForExtraDraw()`, che copia per riferimento le liste `topCardList` e `topBuildingList` nei campi `extraDrawCardSnapshot` e `extraDrawBuildingSnapshot`. Successivamente, `triggerEndRoundBuilding()` applica gli effetti di fine round: se il giocatore possiede *DrawOneMoreCard*, la `ServerVirtualView` del solo proprietario invia `AskExtraDrawMessage(snapshotCards, snapshotBuildings)` in modo *unicast*. Il mercato viene quindi aggiornato normalmente in broadcast per tutti i client.

7.2 Selezione per reference equality nelle liste live

Quando il client risponde con `SelectExtraCardMessage(PlayerDTO, CARD_TYPE, position)`, il parametro `position` indica la posizione nella lista snapshot ricevuta. Il metodo `selectExtraCardFromSnapshot(position, player)` recupera l'oggetto carta dallo snapshot con `snapshot.get(position)` e poi cerca quell'esatto oggetto — per *reference equality* tramite `indexOf()` — nelle liste live del mercato, che nel frattempo potrebbero essere già state aggiornate:

- per le carte tribù: cerca prima in `bottomCardList` (caso normale: le carte sono state spostate dallo shift), poi in `topCardList` (fallback: shift non ancora eseguito);
- per gli edifici: cerca prima in `topBuildingList` (caso normale: gli edifici rimangono in top se non c'è cambio era), poi in `bottomBuildingList` (fallback: cambio era avvenuto).

La rimozione avviene dalla lista live trovata e il client riceve la notifica `BottomCardRemovedMessage` o `TopCardRemovedMessage` con la posizione live della carta, non la posizione snapshot.

7.3 Skip e no-op

Se il giocatore non desidera pescare (ad esempio perché tutte le carte dello snapshot sono eventi, non selezionabili), invia `SkipExtraDrawMessage`. Il server chiama `skipExtraDraw()`, che non esegue alcuna azione: il round prosegue normalmente verso la fase di piazzamento successiva.

Classi e messaggi di riferimento

```
server/model/Game/Market.java           (snapshotForExtraDraw,
selectExtraCardFromSnapshot, buyExtraBuildingFromSnapshot)
server/webLayer/Socket/messages/drawOneMoreCard/AskExtraDrawMessage.java
client/webLayer/Socket/messages/SelectExtraCardMessage.java
client/webLayer/Socket/messages/SkipExtraDrawMessage.java
server/model/Effect/Building/DrawOneMoreCard.java
```

8. Fine partita

Tipo: Sequenziale — Terminazione

Descrizione e motivazione della scelta

La fine partita è innescata dall'esaurimento del mazzo dell'ultima era. Si è scelto di mantenere la struttura del penultimo round identica a quella di un round normale, differenziandola solo tramite le fasi `LAST_ROUND_PLACING_PHASE` e `LAST_ROUND_RESOLVE_ACTION`: in questo modo il codice applicativo lato client non richiede logica speciale per l'ultimo round.

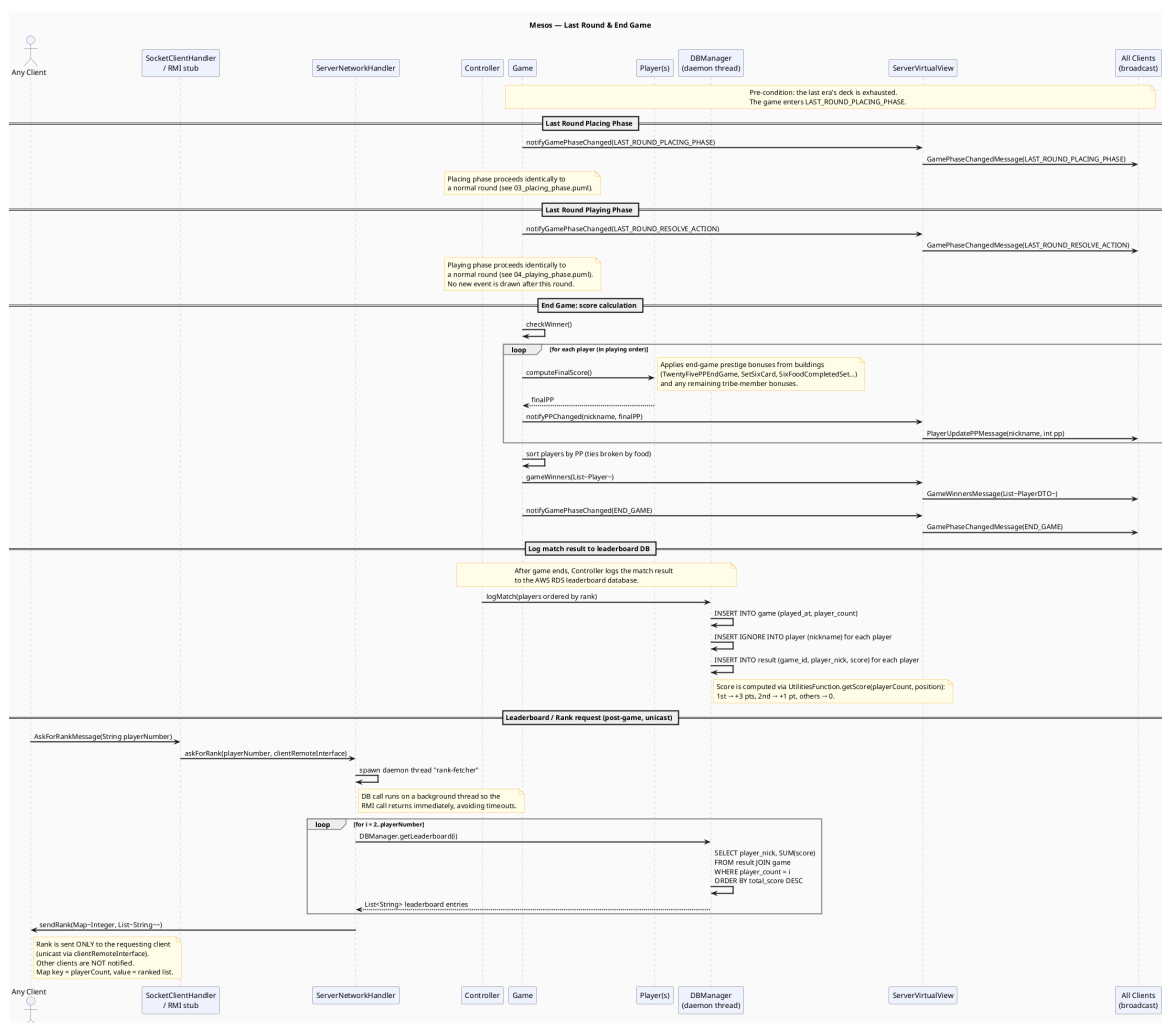


Figura 8.1: Sequenza di fine partita e registrazione del risultato.
Allegato 11 — `allegato_11_fine_partita.pd`

8.1 Calcolo del punteggio finale

Al termine dell'ultimo round il modello chiama `checkWinner()`, che invoca `computeFinalScore()` su ogni giocatore nell'ordine di turno. Questo metodo applica i bonus di fine partita degli edifici posseduti (`TwentyFivePPEndGame`, `SetSixCard`, `SixFoodCompletedSet`, ecc.) e aggiorna

i punti prestigio. Dopo ogni calcolo individuale il server invia `PlayerUpdatePPMessage` in broadcast, così che tutti i client vedano il punteggio finale di ciascun giocatore man mano che viene calcolato.

8.2 Notifica vincitori e transizione END_GAME

Ordinati i giocatori per punti prestigio (pareggi risolti a favore di chi ha più cibo), il server invia in broadcast `GameWinnersMessage(List<PlayerDTO>)`, seguito da `GamePhaseChangedMessage(END_GAME)`. Contestualmente, il `Controller` chiama `DBManager.logMatch()` per registrare il risultato su database (Capitolo 13).

8.3 Richiesta classifica: unicast post-partita

Dopo aver ricevuto `GamePhaseChangedMessage(END_GAME)`, ogni client può inviare `AskForRankMessage` per ricevere la classifica globale. Il server avvia un thread daemon separato per la query al database, evitando di bloccare il thread RMI durante la connessione. La risposta (`SendRankMessage`) è inviata solo al client richiedente tramite `clientRemoteInterface` passata come parametro nella chiamata, senza notificare gli altri giocatori.

Classi e messaggi di riferimento

```
server/webLayer/Socket/messages/gameMessages/GameWinnersMessage.java
server/webLayer/Socket/messages/gameMessages/GamePhaseChangedMessage.java
client/webLayer/Socket/messages/askForRankMessage.java
server/webLayer/Socket/messages/SendRankMessage.java
server/model/DBmanager/DBManager.java
server/model/Effect/Building/TwentyFivePPEndGame.java
```

9. Heartbeat — Ping/Pong

Tipo: Infrastrutturale — Resilienza

Descrizione e motivazione della scelta

Il sistema di heartbeat è bidirezionale e indipendente per le due direzioni: il client invia ping al server ogni secondo; il server invia pong al client ogni secondo. Si è valutato necessario mantenere entrambe le direzioni per rilevare condizioni di guasto distinte: il ping rileva la morte del client (o la sua assenza); il pong rileva la morte del server. Entrambi i meccanismi si basano su soglie configurabili, mantenendo il sistema reattivo anche su connessioni instabili.

9.1 Ping client → server

Il client invia `PingMessage(PlayerDT0)` ogni circa un secondo tramite un task schedulato. Lato server, la ricezione viene gestita da `ServerNetworkHandler.ping()`, che recupera la `ServerVirtualView` corrispondente al nickname e chiama `view.receivePing()`: questo azzerava il contatore atomico `missedPings` tramite `AtomicInteger.set(0)`, garantendo correttezza in contesto multi-thread senza lock globale.

9.2 Pong server → client e Heartbeat Watchdog

Il **Heartbeat Watchdog** è uno `ScheduledExecutorService` che ogni secondo esegue due callback distinte su `ServerNetworkHandler`: `sendPongs()` invia `PongMessage` a ogni `ServerVirtualView` connessa; `checkMissedPings()` incrementa il contatore `missedPings` di ogni view e, se supera la soglia di 5 (circa cinque secondi di silenzio), invoca `notifyPlayerDisconnected(nickname)` (Capitolo 10).

9.3 PongWatchdog lato client

Il **PongWatchdog** è un thread daemon sul client che controlla ogni secondo se sono trascorsi più di 5000 ms dall'ultimo messaggio ricevuto. A differenza della versione server, lato Socket qualsiasi `ServerToClientMessage` in arrivo (non solo il pong esplicito) aggiorna il timestamp di ultima attività: in questo modo eventuali burst di messaggi di gioco mantengono il watchdog silenzioso senza richiedere pong aggiuntivi. Con RMI, invece, solo la chiamata `pong()` aggiorna il timer, in quanto non vi è un flusso di messaggi unificato su cui appoggiarsi.

Classi e messaggi di riferimento

```
client/webLayer/Socket/messages/PingMessage.java
server/webLayer/Socket/messages/gameMessages/PongMessage.java
client/webLayer/PongWatchdog.java
server/webLayer/RMI/ServerNetworkHandler.java      (ping,      sendPongs,
checkMissedPings)
server/webLayer/ServerVirtualView.java (receivePing, incrementMissedPings)
```

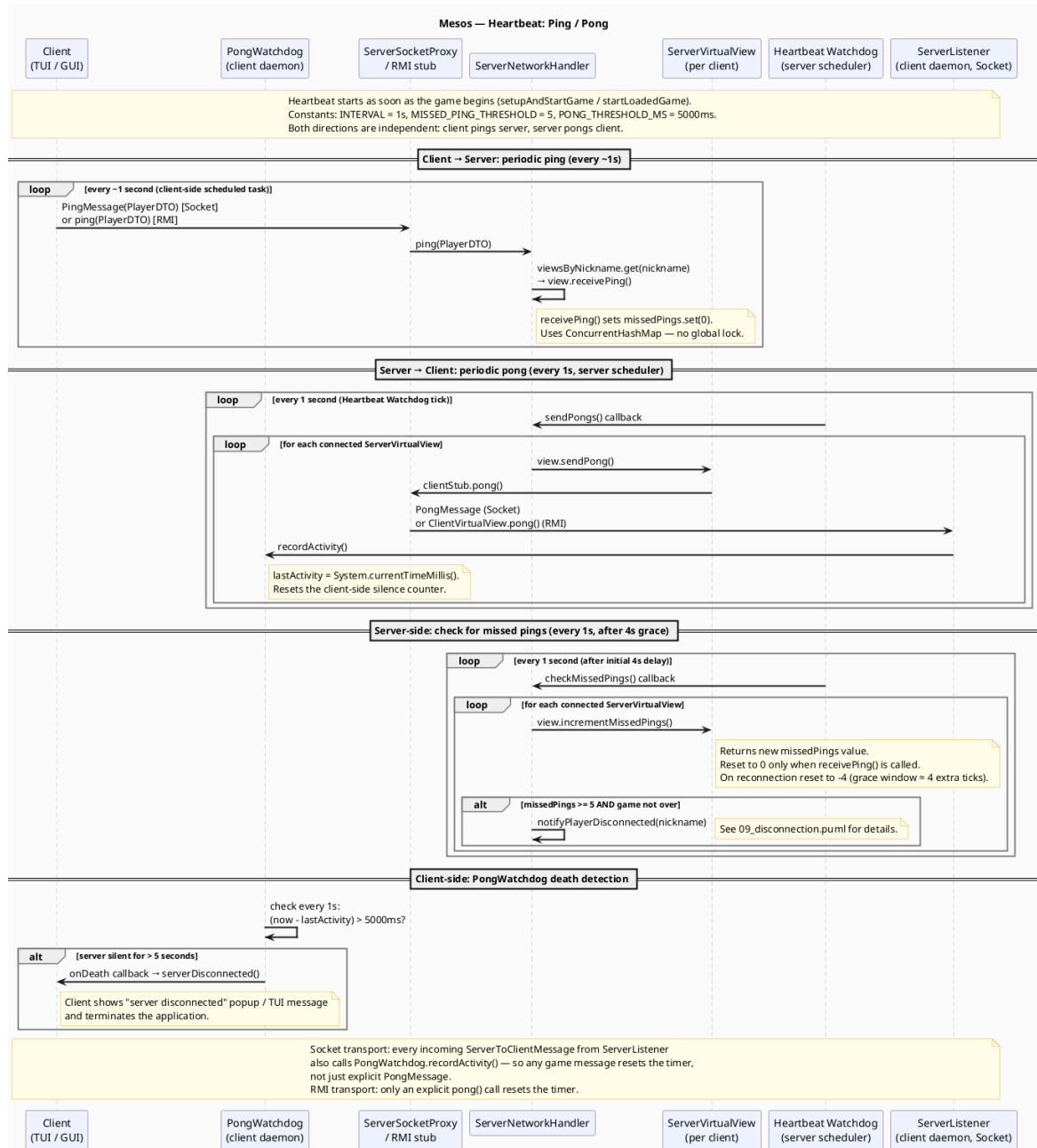


Figura 9.1: Sequenza del meccanismo di heartbeat. Allegato 12 — allegato_12_heartbeat.pdf

10. Disconnessione

Tipo: Infrastrutturale — Resilienza

Descrizione e motivazione della scelta

Una disconnessione può manifestarsi in tre modi distinti a seconda del trasporto e del momento in cui si verifica. Si è scelto di far convergere tutti e tre i percorsi verso un unico handler `notifyPlayerDisconnected(nickname)` marcato `synchronized`, così da garantire che la procedura di disconnessione sia eseguita al più una volta per giocatore, anche se più eventi si verificano in rapida successione.

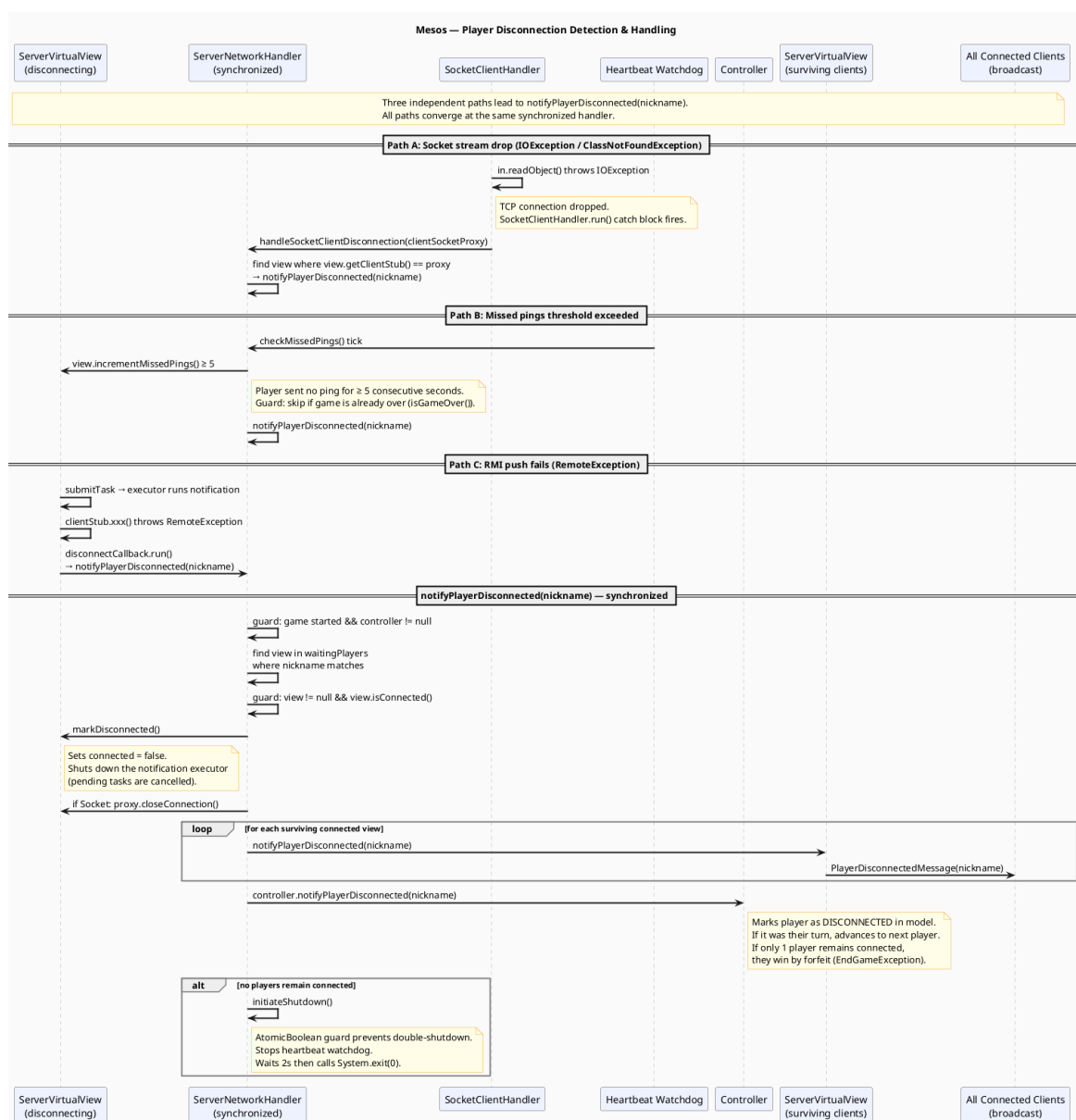


Figura 10.1: Sequenza di rilevamento e gestione della disconnessione.

Allegato 13 — `allegato_13_disconnessione.pdf`

10.1 I tre percorsi di rilevamento

Percorso A — IOException su Socket: il `SocketClientHandler` rileva l'eccezione nella chiamata bloccante `in.readObject()` e chiama `handleSocketClientDisconnection(proxy)`: il server identifica la `ServerVirtualView` corrispondente al proxy e invoca `notifyPlayerDisconnected()`.

Percorso B — Soglia missedPings: il `Heartbeat Watchdog` rileva che un giocatore non ha inviato ping per cinque secondi consecutivi (Capitolo 9) e chiama direttamente `notifyPlayerDisconnected()` sul `ServerNetworkHandler`.

Percorso C — RemoteException su RMI: la `ServerVirtualView` rileva un errore durante l'invio di una notifica al client. Il meccanismo di retry di `ThrowingRunnable/submitTask()` (descritto nel report dei design pattern, Capitolo 11) esaurisce i tentativi e chiama il `disconnectCallback`, che a sua volta invoca `notifyPlayerDisconnected()`.

10.2 Handler sincronizzato e effetti sulla partita

All'interno di `notifyPlayerDisconnected()`, protetto da `synchronized`, vengono eseguiti: la verifica che il giocatore sia effettivamente ancora connesso (`view.isConnected()`), evitando doppia esecuzione; la chiamata a `markDisconnected()` sulla view, che imposta `connected = false` e interrompe l'`ExecutorService` di notifica; l'invio di `PlayerDisconnectedMessage` a tutti i client ancora connessi; infine, la notifica al `Controller`, che marca il giocatore come `DISCONNECTED` nel modello e, se era il suo turno, avanza al giocatore successivo. Se rimane un solo giocatore connesso, viene dichiarato vincitore per abbandono degli avversari tramite `EndGameException`.

Classi e messaggi di riferimento

<code>server/webLayer/RMI/ServerNetworkHandler.java</code>	(<code>notifyPlayerDisconnected</code> , <code>handleSocketClientDisconnection</code>)
<code>server/webLayer/ServerVirtualView.java</code>	(<code>markDisconnected</code> , <code>incrementMissedPings</code>)
<code>server/webLayer/Socket/messages/gameMessages/PlayerDisconnectedMessage.java</code>	
<code>server/model/Controller/Controller.java</code>	(<code>notifyPlayerDisconnected</code>)

11. Riconnessione

Tipo: Infrastrutturale — Resilienza

Descrizione e motivazione della scelta

Il meccanismo di riconnessione consente a un giocatore che si è disconnesso di rientrare in una partita in corso. Il giocatore si riconnette semplicemente usando lo stesso nickname, esattamente come durante la fase di lobby.

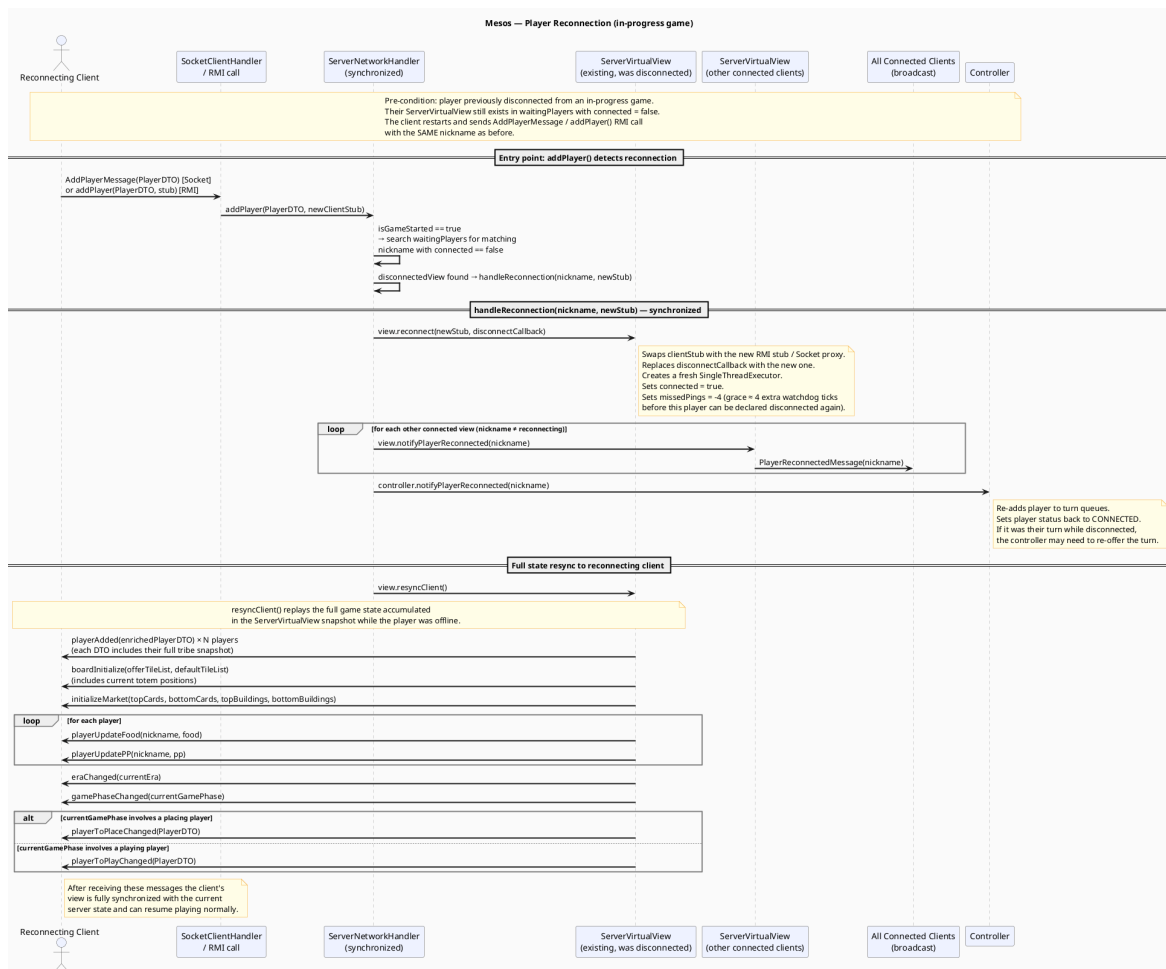


Figura 11.1: Sequenza di riconnessione e risincronizzazione dello stato.
Allegato 14 — allegato_14_riconnessione.pdf

11.1 Rilevamento e swap dello stub

Quando il server riceve `AddPlayerMessage` o la chiamata RMI `addPlayer()` mentre una partita è in corso (`isGameStarted == true`), cerca nelle `waitingPlayers` una `ServerVirtualView` con nickname corrispondente e `connected == false`. Se la trova, invoca `handleReconnection(nickname, newStub)`, che a sua volta chiama `view.reconnect(newStub, disconnectCallback)`: questo sostituisce il vecchio `clientStub` con il nuovo (RMI stub o `ClientSocketProxy` aggior-

nato), rimpiazza il `disconnectCallback`, crea un nuovo `SingleThreadExecutor` (quello precedente era stato interrotto dalla disconnessione) e ripristina il flag `connected = true`.

11.2 Grace window sul watchdog

Al momento della riconnessione, `missedPings` viene impostato a `-4` anziché a `0`. Questo crea una finestra di grazia di circa quattro secondi durante la quale il watchdog non dichiara immediatamente disconnesso il giocatore appena rientrato, anche se il primo ping non arriva esattamente dopo un secondo. Si è valutato che questa tolleranza sia necessaria per assorbire la latenza iniziale di riconnessione, specialmente su reti instabili.

11.3 Resync completo dello stato

Dopo aver notificato tutti i client ancora connessi tramite `PlayerReconnectedMessage` e aggiornato il `Controller`, il server chiama `view.resyncClient()`, che replica l'intero stato di gioco al client riconnesso basandosi sullo snapshot locale della `ServerVirtualView`:

- `playerAdded(enrichedPlayerDTO)` per ogni giocatore, con la tribù completa inclusa;
- `boardInitialize(offerTileList, defaultTileList)`, con le posizioni correnti dei totem;
- `initializeMarket(topCards, bottomCards, topBuildings, bottomBuildings)`;
- `playerUpdateFood()` e `playerUpdatePP()` per ogni giocatore;
- `eraChanged(currentEra)` e `gamePhaseChanged(currentPhase)`;
- `playerToPlaceChanged()` o `playerToPlayChanged()` in base alla fase corrente.

Al termine del resync il client è completamente allineato con il server e può riprendere a giocare normalmente.

Classi e messaggi di riferimento

```
server/webLayer/RMI/ServerNetworkHandler.java (handleReconnection)
server/webLayer/ServerVirtualView.java (reconnect, resyncClient)
server/webLayer/Socket/messages/gameMessages/PlayerReconnectedMessage.java
server/model/Controller/Controller.java (notifyPlayerReconnected)
```

12. Caricamento partita salvata

Tipo: Infrastrutturale — Persistenza

Descrizione e motivazione della scelta

Il caricamento di una partita salvata differisce dalla lobby normale perché il modello non viene creato da zero, bensì ripristinato da un memento su disco. Si è scelto di distinguere i due flussi tramite messaggi dedicati (`LoadGameMessage` e `JoinGameLoadedMessage`) anziché riutilizzare quelli di lobby, così da rendere esplicita al server la necessità di caricare il memento anziché creare un nuovo modello.

12.1 Fase 1: il primo giocatore carica la partita

Il giocatore che avvia il caricamento invia `LoadGameMessage`. Il server verifica che non sia già in corso una partita (`controller == null`), crea un nuovo `Controller` e chiama `loadGame()`, che legge il memento da disco, ripristina il modello (giocatori, tribù, mercato, board, era, fase) e valida che il nickname del richiedente sia presente nel salvataggio. Il server crea la `ServerVirtualView` per il primo giocatore e rimane in attesa degli altri, memorizzando il numero di giocatori attesi da `controller.getAllPlayers().size()`.

12.2 Fase 2: i giocatori rimanenti si uniscono

Ciascun giocatore rimanente invia `JoinGameLoadedMessage`. Il server chiama `textttcontroller.reconnectLoadedPlayer(player)`, che valida il nickname e lancia `IllegalStateException` se lo stesso nickname si connette due volte. Quando si connette l'ultimo giocatore atteso, viene lanciata internamente `GameReadyToStartException` e il server chiama `startLoadedGame()`.

12.3 Fase 3: `startLoadedGame()` e `resumeGame()`

`startLoadedGame()` lega tutte le `ServerVirtualView` come observer, esegue `crossRegisterPlayerObservers()`, avvia il watchdog heartbeat e chiama `forceInitialPlayersSync()` (con le stesse motivazioni descritte nel Capitolo 3). Infine chiama `controller.resumeGame()`, che ripristina lo stato del modello dagli snapshot e ri-spara tutte le notifiche observer in sequenza, producendo un flusso di messaggi identico a quello di un resync completo: `boardInitialize()`, `initializeMarket()`, `playerUpdateFood/PP()` per ogni giocatore, `eraChanged()`, `gamePhaseChanged()` e infine `playerToPlaceChanged()` o `playerToPlayChanged()`. Il gioco riprende esattamente dal punto in cui era stato salvato.

Classi e messaggi di riferimento

```
client/webLayer/Socket/messages/LoadGameMessage.java
client/webLayer/Socket/messages/JoinGameLoadedMessage.java
server/webLayer/RMI/ServerNetworkHandler.java (loadGame, joinGameLoaded,
startLoadedGame)
```

```
server/model/Controller/Controller.java (loadGame, reconnectLoadedPlayer,  
resumeGame)  
server/model/persistence/PersistenceLogger.java
```

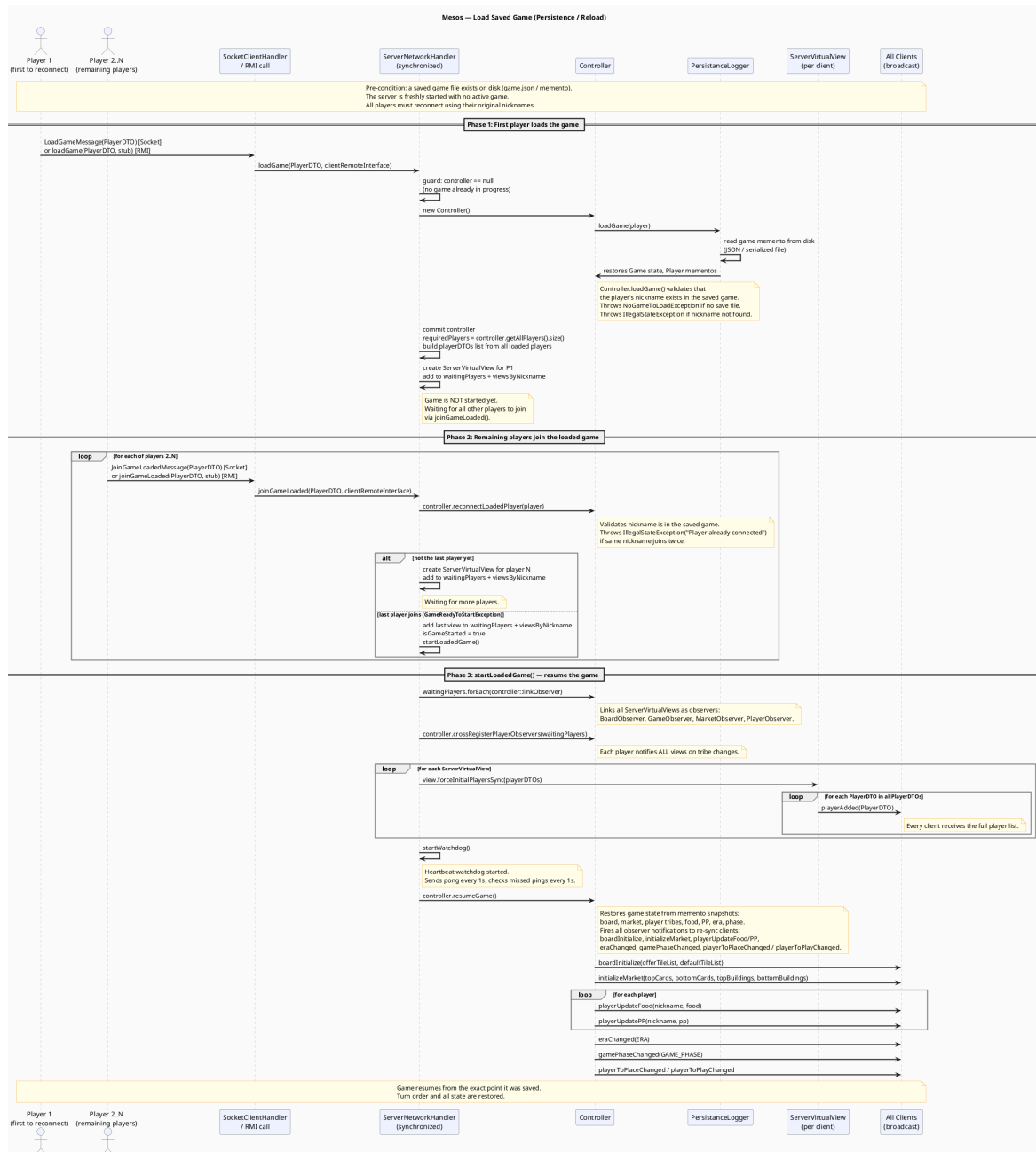


Figura 12.1: Sequenza di caricamento di una partita salvata.
 Allegato 15 — allegato_15_caricamento_partita.pdf

13. Database e classifica

Tipo: Infrastrutturale — Persistenza

Descrizione e motivazione della scelta

Il requisito avanzato di classifica richiede di mantenere uno storico persistente dei risultati su database esterno. Si è scelto di utilizzare un'istanza MySQL su AWS RDS, accessibile tramite JDBC, con una connessione statica riutilizzata attraverso tutte le chiamate per minimizzare l'overhead di connessione. Tutte le operazioni sul database vengono eseguite su thread daemon dedicati per evitare di bloccare i thread RMI durante le query.

13.1 Schema del database

Il database `leaderboardDB` è composto da tre tabelle: `game(id, played_at, player_count)` registra ogni partita con data e numero di giocatori (PK: `id`); `player(nickname)` mantiene un registro unico dei giocatori (PK: `nickname`); `result(game_id, player_nick, score)` associa ogni partita ai giocatori con il punteggio assegnato (PK: `game_id, player_nick`). Il punteggio è calcolato tramite `UtilitiesFunction.getScore(playerCount, position)`.

13.2 Registrazione del risultato: `logMatch()`

Al termine di ogni partita il `Controller` chiama `DBManager.logMatch(List<Player>)`, dove la lista è ordinata dal primo all'ultimo classificato. Il `DBManager` apre o riutilizza la connessione JDBC, inserisce un record in `game`, poi per ogni giocatore esegue `INSERT IGNORE INTO player` (idempotente: non duplica giocatori già presenti) e `INSERT INTO result`. Si è scelto di eseguire `logMatch()` su thread daemon per non bloccare il flusso principale: in caso di irraggiungibilità del database l'errore viene loggato ma la partita prosegue normalmente.

13.3 Recupero della classifica: `getLeaderboard()`

Quando un client invia `AskForRankMessage`, il `ServerNetworkHandler` avvia un thread daemon *rank-fetcher* che esegue `getLeaderboard(i)` per *i* da 2 al numero di giocatori della partita. La query raggruppa i risultati per `player_nick` e ne somma i punteggi, restituendo la classifica ordinata. La risposta `SendRankMessage(Map<Integer, List<String>)` viene inviata in modalità *unicast* esclusivamente al client richiedente, tramite l'interfaccia `clientRemoteInterface` passata come parametro nella chiamata originale. Se il database non è raggiungibile, `getLeaderboard()` restituisce `List.of("Classifica non disponibile")`.

Classi e messaggi di riferimento

```
server/model/DBmanager/DBManager.java
server/webLayer/RMI/ServerNetworkHandler.java (askForRank)
client/webLayer/Socket/messages/askForRankMessage.java
server/webLayer/Socket/messages/SendRankMessage.java
```

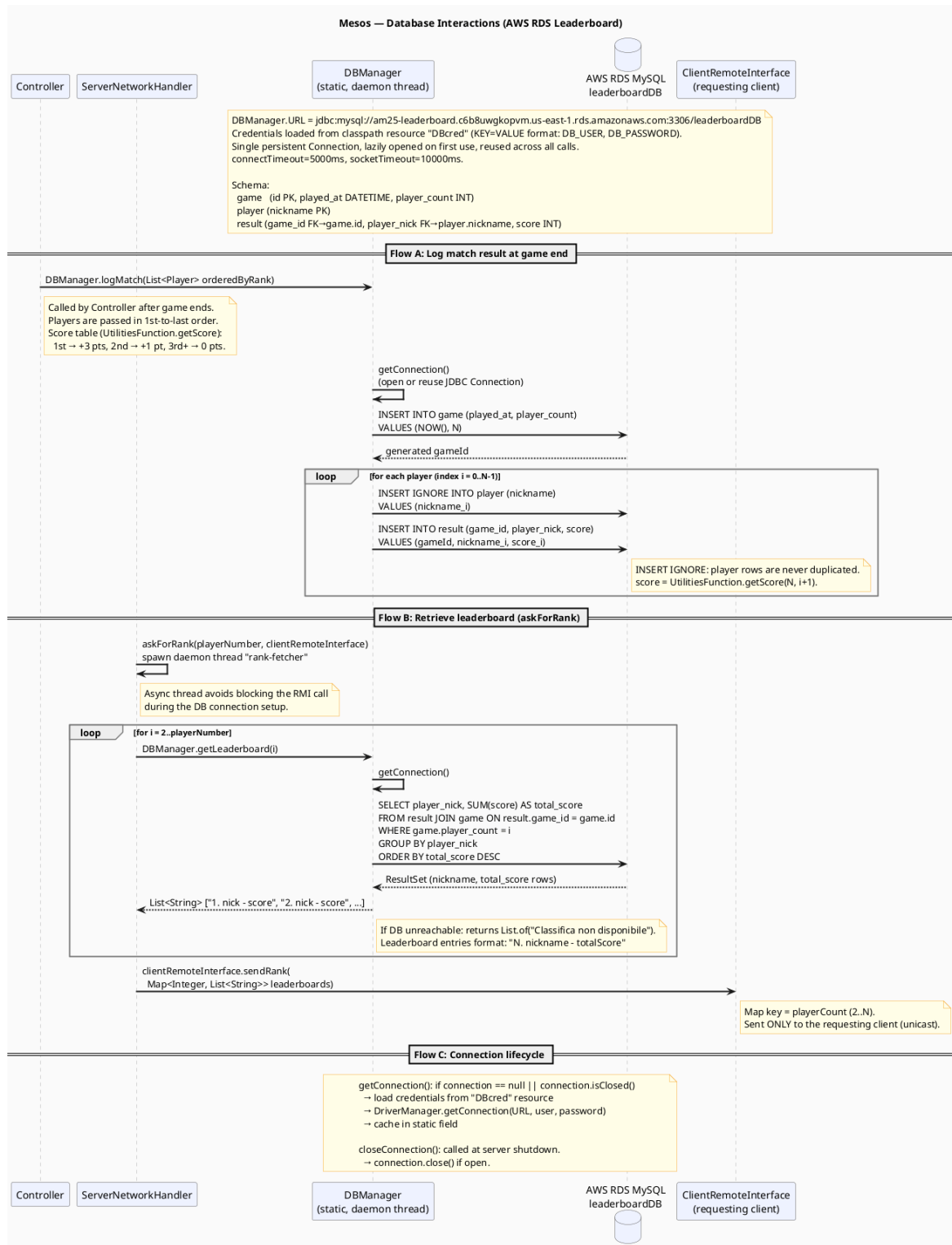


Figura 13.1: Sequenza delle interazioni con il database AWS RDS.

Allegato 16 — allegato_16_database.pdf

Considerazioni conclusive

Le scelte progettuali che governano il protocollo di comunicazione non sono state adottate in modo isolato, ma in stretta sinergia con i design pattern architetturali descritti nel report di analisi del codice. La tabella seguente riassume le relazioni più significative tra decisioni di protocollo e scelte progettuali.

Aspetto	Considerazione
Trasparenza del trasporto	Il pattern Proxy (Capitolo 5 del report Design Pattern) rende il codice applicativo ignaro del protocollo di rete sottostante. L'unica differenza visibile è la creazione iniziale dello stub: lookup nel registry RMI oppure costruzione del <code>ServerSocketProxy</code> .
Messaggi come Command	Il pattern Command (Capitolo 6) consente al dispatcher di essere completamente polimorfico: ogni messaggio sa auto-eseguirsi sull'interfaccia remota di destinazione. L'aggiunta di nuovi tipi di messaggio durante lo sviluppo (heartbeat, extra draw, classifica) non ha richiesto modifiche ai dispatcher esistenti.
Notifiche tramite Observer	Il pattern Observer (Capitolo 1) disaccoppia il modello dalla rete. La <code>ServerVirtualView</code> è il punto di raccolta che riceve tutti gli eventi di dominio e li traduce in messaggi di rete. Il meccanismo di retry tramite <code>ThrowingRunnable</code> garantisce la consegna anche su connessioni instabili.
Snapshot DTO per il resync	Il pattern DTO (Capitolo 9) è indispensabile per la riconnessione: la <code>ServerVirtualView</code> mantiene uno snapshot locale di tutti gli oggetti di dominio serializzati, permettendo di ricostruire lo stato completo senza interrogare il modello. Senza questo snapshot, un resync coerente in fase di partita attiva sarebbe impossibile senza blocchi di mutua esclusione sul modello.
Resilienza a tre livelli	Il sistema di resilienza alle disconnessioni opera a tre livelli indipendenti: rilevamento (tre percorsi convergenti verso <code>notifyPlayerDisconnected</code>), recupero (handler <code>synchronized</code> con grace window), risincronizzazione (<code>resyncClient</code> basato sullo snapshot DTO). Questa separazione rende ogni livello sostituibile o estendibile indipendentemente dagli altri.
Persistenza non invasiva	Il pattern Memento (Capitolo 4) consente di salvare lo stato senza esporre i dettagli interni del modello. Il protocollo di caricamento (<code>LoadGameMessage</code> / <code>JoinGameLoadedMessage</code>) rispecchia questa separazione: il network layer si limita ad aggregare i giocatori e a invocare <code>resumeGame()</code> , senza conoscere nulla della struttura del memento.

A. Architettura di rete — diagramma completo

Il diagramma allegato riporta la panoramica completa dell'architettura di rete del sistema, con tutti i componenti del Client JVM e del Server JVM, i due canali di trasporto (Socket TCP/IP e RMI), il flusso dello stub `ClientRemoteInterface` passato in-band, il meccanismo del Heartbeat Watchdog e le interazioni con il database AWS RDS.

Allegato 1 — `allegato_01_architettura_di_rete.pdf`

Componenti Client JVM, Server JVM, canale TCP/IP, registro RMI, stub in-band, Heartbeat Watchdog e DBManager.

B. Gerarchia completa dei messaggi

Le due famiglie di messaggi del protocollo sono descritte nella loro interezza tramite diagrammi UML di classe allegati come file separati. Ogni package raggruppa i messaggi per fase o funzionalità; tutte le classi implementano `java.io.Serializable` tramite la rispettiva interfaccia radice, garantendo la compatibilità con la serializzazione Java usata dal trasporto Socket. Su RMI, gli stessi metodi dell'interfaccia `ClientRemoteInterface` svolgono il ruolo equivalente senza serializzazione esplicita.

- **Allegato 4** — `allegato_04_gerarchia_messaggi_c2s.pdf`
Gerarchia di `ClientToServerMessage` suddivisa per fase (Lobby, Persistenza, Piazzamento, Fase di Gioco, Extra Draw, Post-Partita, Heartbeat, Errore).
- **Allegato 5** — `allegato_05_gerarchia_messaggi_s2c.pdf`
Gerarchia di `ServerToClientMessage` suddivisa per categoria funzionale (Stato Partita, Board, Mercato, Giocatore, Extra Draw, Classifica, Connettività, Heartbeat).

C. Fase di gioco — diagramma completo

Il diagramma allegato riporta la sequenza completa della fase **RESOLVE_ACTION**, con tutte le varianti di selezione: carta tribù dalla riga superiore, acquisto edificio dalla riga superiore, carta tribù dalla riga inferiore, acquisto edificio dalla riga inferiore e passaggio. Per ciascuna variante sono mostrati i messaggi broadcast verso tutti i client e la logica di avanzamento del turno.

Allegato 8 — `allegato_08_fase_di_gioco.pdf`

Sequenza completa **RESOLVE_ACTION** con tutte le varianti di selezione dal mercato e avanzamento turno/round.

D. Fine round — risoluzione eventi: diagramma completo

Il diagramma allegato riporta la sequenza completa di fine round: ritorno ai default tile con `OrderOnDefaultTileMessage`, risoluzione di tutti e quattro i tipi di evento (`HUNT`, `PAINTINGS`, `SHAMANIC_RIT`, `SUSTENANCE`) con i relativi messaggi di aggiornamento cibo e punti prestigio, aggiornamento del mercato nei tre scenari (nessun cambio era, cambio era, mazzo esaurito) e salvataggio su disco.

Allegato 9 — `allegato_09_risoluzione_eventi.pdf`

Sequenza completa di fine round: ritorno ai tile, risoluzione eventi, aggiornamento mercato (tre scenari) e salvataggio su disco.